

EDITORA AGÊNTICA

Mini-guia: Harness Engineering — Do Modelo ao Sistema Autônomo Confiável

O primeiro passo de Harness Engineering — Do Modelo ao Sistema Autônomo Confiável, do início ao fim

PARA INICIANTES

Heverton Eduardo Peres

Um recorte de harness-engineering

A Revolução dos Agentes: Por Que o Modelo Não Basta

Por que esta etapa existe

Explicar por que LLMs sozinhos não produzem trabalho confiável e introduzir a equação $\text{Agente} = \text{Modelo} + \text{Harness}$, com o mapa do que será construído na obra.

O que você vai produzir



AGENTS.md

Passo a passo

O Modelo Puro vs. o Agente com Harness

```
"""Demonstra a diferenca entre LLM puro e agente com harness minimo."""

from __future__ import annotations

class LLM:
    """Simulacao de um modelo de linguagem."""

    def responder(self, prompt: str) -> str:
        # Em producao isto seria uma chamada real a um provedor de LLM.
        if "soma" in prompt.lower():
            return "4" # resposta plausivel, porem errada para este caso
        return "Nao entendi o pedido."

class HarnessMinimo:
    """Harness minimo: ferramenta + teste + limite de tentativas."""

    def __init__(self, modelo: LLM) -> None:
        self.modelo = modelo
        self.tentativas = 0

    def executar(self, prompt: str, max_tentativas: int = 3) -> str:
        self.tentativas = 0
        while self.tentativas < max_tentativas:
            self.tentativas += 1
            resposta = self.modelo.responder(prompt)
            if self._testar(resposta, prompt):
                return resposta
        raise RuntimeError("harness: limite de tentativas excedido")

    def _testar(self, resposta: str, prompt: str) -> bool:
        # Test harness deterministico: valida o contrato antes de aceitar.
        if "soma 2+2" in prompt.lower():
            return resposta.strip() == "4"
```

```
        return bool(resposta.strip())

def main() -> None:
    modelo = LLM()
    prompt = "Quanto e 2+2? (soma 2+2)"

    # LLM puro: aceita qualquer resposta como verdade.
    resposta_pura = modelo.responder(prompt)
    print(f"LLM puro devolveu: {resposta_pura} (aceita sem verificacao)")

    # Agente com harness: a resposta passa por teste deterministico.
    ag
```

A Simulação do Agente Sem Harness Quebrando

```
"""Agente sem guardrail vs. agente com guardrail de aprovacao."""

class Banco:
    def __init__(self) -> None:
        self.tabelas = ["clientes", "pedidos", "produtos"]

    def apagar_tabela(self, nome: str) -> None:
        if nome in self.tabelas:
            self.tabelas.remove(nome)

def agente_sem_guardrail(banco: Banco, pedido: str) -> str:
    # Executa qualquer acao sem verificacao humana.
    if "apagar" in pedido:
        alvo = pedido.split("apagar")[-1].strip()
        banco.apagar_tabela(alvo)
        return f"tabela {alvo} apagada (sem aprovacao)"
    return "nada feito"

def agente_com_guardrail(banco: Banco, pedido: str, aprovado: bool = False) -> str:
    # Acoes destrutivas exigem aprovacao explicita (approval gate).
    if "apagar" in pedido:
        alvo = pedido.split("apagar")[-1].strip()
        if not aprovado:
            return f"BLOQUEADO: apagar {alvo} requer aprovacao humana"
        banco.apagar_tabela(alvo)
        return f"tabela {alvo} apagada (aprovado)"
    return "nada feito"

def main() -> None:
    b1 = Banco()
    b2 = Banco()
    pedido = "apagar clientes"
```

```
resultado_sem = agente_sem_guardrail(b1, pedido)
resultado_com = agente_com_guardrail(b2, pedido, aprovado=False)

print(f"Sem harness : {resultado_sem} | tabelas restantes: {b1.tabelas}")
print(f"Com harness: {resultado_com} | tabelas restantes: {b2.tabelas}")

if __name__ == "__main__":
    main()
```

Está pronto quando:

- ☐ ☐ Exercício 1 — Inventário do arnês.** Liste os cinco componentes de um harness (âncora
- ☐ ☐ Escreva uma frase dizendo qual falha do agente ele evita
- ☐ ☐ Use uma tabela como a abaixo
- ☐ ☐ Complete a função `executar_acao` para que o harness valide a ação antes de entregá-la ao modelo — a lição central do capítulo: quem executa é o harness
- ☐ ☐ Exercício 3 — Diagnóstico.** Um agente de suporte apagou um arquivo de produção porque o prompt do sistema dizia “você tem autonomia total”
- ☐ ☐ Aponte: (a) qual componente do harness deveria ter impedido
- ☐ ☐ (b) qual evidência a trilha deve conter para o pós-incidente

Armadilhas desta etapa

- ☐ “O modelo é tão bom que não precisa de teste”**: o DORA 2024 mostrou exatamente o contrário — produtividade individual sem estabilidade de entrega é um custo escondido [9]
- ☐ Permissão ampla “só por enquanto”**: tokens com escopo global e diretórios liberados são o vetor favorito de incidentes [17]
- ☐ Autonomia total sem approval gates**: cancelar a confirmação humana “para acelerar” transfere o risco de erro para a escala — um erro repetido 100 vezes não é 100 vezes mais rápido, é 100 vezes mais caro [16]
- ☐ Sem observabilidade**: agente que faz muito e não deixa rastro é um passivo de auditoria ambulante [12]

- ☐ Comprar o hype do “agente pronto”**: o relatório da LangChain com mais de 1.300 profissionais mostra que 57% das organizações já têm agentes em produção — mas também que observabilidade e evals, as fundações do harness, ainda são os itens menos maduros [12]

Próximo passo

Este material é um recorte de **Harness Engineering — Do Modelo ao Sistema Autônomo Confiável**. A obra completa traz a teoria, os exemplos comentados e as referências.

Quero a obra completa